# Tata Cara Push dan Pull pada GitHub untuk Pengelolaan Versi Kode Sumber: Panduan Praktis dengan Studi Kasus Website Portfolio

*Push and Pull Procedures on GitHub for Source Code Version Management: A Practical Guide with a Portfolio Website Case Study*

Muhamad Faridzqi Suryadi
*Program Studi Teknik Informatika, Universitas Nahdlatul Ulama Indomesia*
*Repositori: github.com/faridSrydi/github-pcd*

***Abstrak***

Pengelolaan versi kode sumber merupakan kebutuhan dasar dalam pengembangan perangkat lunak, baik secara individu maupun tim. Git sebagai sistem kontrol versi terdistribusi dan GitHub sebagai layanan hosting repositori telah menjadi kombinasi yang paling umum digunakan, tetapi pengguna pemula sering mengalami kendala pada operasi *push* dan *pull*, seperti kegagalan autentikasi, penolakan *push*, dan konflik penggabungan. Artikel ini menyajikan prosedur lengkap penggunaan *push* dan *pull* pada GitHub, mulai dari instalasi dan konfigurasi Git, autentikasi, pengiriman pertama ke repositori kosong, siklus kerja harian, pengambilan perubahan dengan *fetch*, *pull*, dan *clone*, penggunaan *branch* dan *pull request*, hingga penanganan konflik. Prosedur tersebut diilustrasikan melalui studi kasus pembuatan website portfolio statis berbasis HTML, CSS, dan JavaScript yang disimpan pada repositori github-pcd dan dipublikasikan melalui GitHub Pages. Setiap perintah dijelaskan fungsi dan alasan penggunaannya, dilengkapi diagram alur, contoh keluaran terminal, dan tabel pemecahan masalah. Hasilnya berupa panduan terstruktur yang dapat dijadikan acuan oleh mahasiswa dan pengembang pemula untuk bekerja dengan GitHub secara aman, rapi, dan terdokumentasi.

**Kata kunci:** *Git, GitHub, push, pull, kontrol versi, GitHub Pages*

# I. PENDAHULUAN

## A. Latar Belakang

Pengembangan perangkat lunak melibatkan perubahan berkelanjutan pada kode sumber. Tanpa sistem kontrol versi, perubahan tersebut sulit dilacak, dibatalkan, atau digabungkan ketika dikerjakan oleh lebih dari satu orang [1], [2]. Sistem kontrol versi mencatat riwayat perubahan sehingga pengembang dapat kembali ke kondisi sebelumnya, membandingkan versi, dan mengerjakan fitur yang berbeda secara paralel.

Git, yang dikembangkan oleh Linus Torvalds pada tahun 2005, adalah sistem kontrol versi terdistribusi: setiap komputer menyimpan salinan lengkap repositori beserta riwayatnya [1], [3]. GitHub menambahkan lapisan layanan di atas Git berupa penyimpanan repositori jarak jauh, *pull request*, pelacakan isu, dan hosting situs statis melalui GitHub Pages [4]. Penggunaan Git dan GitHub juga dianjurkan dalam konteks ilmiah karena meningkatkan transparansi dan reprodusibilitas pekerjaan [5]–[7].

Meskipun demikian, pengguna pemula sering terhambat pada operasi yang menghubungkan repositori lokal dengan repositori jarak jauh, yaitu *push* dan *pull*. Kendala yang umum dijumpai meliputi autentikasi yang gagal karena GitHub tidak lagi menerima kata sandi akun untuk operasi Git melalui HTTPS, penolakan *push* akibat riwayat lokal tertinggal dari *remote*, serta konflik penggabungan. Sebagian besar panduan yang tersedia berbahasa Inggris dan terpisah-pisah sehingga kurang membantu pembelajar yang membutuhkan alur lengkap dari awal.

## B. Tujuan

Tujuan penulisan artikel ini adalah sebagai berikut.

1. Menjelaskan konsep dasar Git dan GitHub yang melandasi operasi *push* dan *pull*.
2. Menyusun prosedur langkah demi langkah untuk *push* pertama ke repositori kosong, siklus kerja harian, *pull*, dan *clone*.
3. Menjelaskan penggunaan *branch*, *pull request*, dan penanganan konflik.
4. Mengilustrasikan seluruh prosedur melalui studi kasus website portfolio yang dipublikasikan dengan GitHub Pages.

## C. Ruang Lingkup

Pembahasan dibatasi pada penggunaan Git melalui antarmuka baris perintah, repositori publik di GitHub, *branch* utama bernama `main`, dan autentikasi melalui HTTPS dengan *personal access token* atau SSH. Fitur lanjutan seperti GitHub Actions, *submodule*, dan manajemen tim berskala besar tidak dibahas.

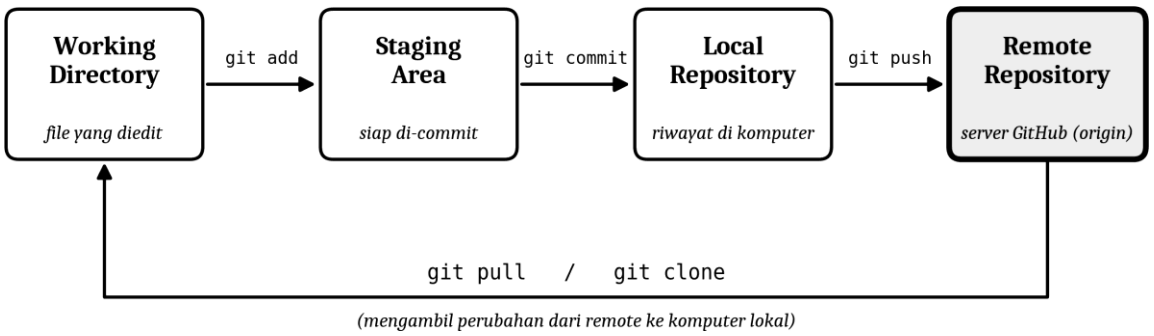
# II. LANDASAN TEORI

## A. Sistem Kontrol Versi

Sistem kontrol versi (*version control system*, VCS) dapat dikelompokkan menjadi tiga jenis: lokal, terpusat, dan terdistribusi [2]. VCS lokal menyimpan riwayat hanya pada satu komputer. VCS terpusat menyimpan riwayat pada satu server sehingga bergantung pada koneksi dan menjadi titik kegagalan tunggal. VCS terdistribusi seperti Git menyalin seluruh riwayat ke setiap klien, sehingga pekerjaan dapat dilakukan tanpa koneksi dan setiap salinan berfungsi sebagai cadangan [1].

## B. Git dan Tiga Area Kerja

Git menyimpan data sebagai rangkaian *snapshot*. Setiap berkas dalam proyek berada pada salah satu dari tiga keadaan, yaitu *modified*, *staged*, atau *committed*, yang berkaitan dengan tiga area: *working directory*, *staging area*, dan repositori [1]. Perubahan harus melewati tahap *add* dan *commit* di komputer lokal sebelum dikirim ke *remote* dengan *push*, sebagaimana ditunjukkan pada Gambar 1.




**Gambar 1.** Alur perubahan berkas pada Git dari *working directory* hingga *remote repository* di GitHub.

## C. GitHub dan Repositori Jarak Jauh

GitHub menyediakan *remote repository* yang dapat diakses melalui internet. Secara konvensi, remote utama diberi nama origin. Selain penyimpanan, GitHub menyediakan *pull request* untuk peninjauan perubahan, pelacakan isu, dan GitHub Pages untuk menyajikan situs statis langsung dari repositori [4].

## D. Konsep Push dan Pull

*Push* mengirim commit lokal yang belum ada di remote. Remote hanya menerima *push* apabila perubahan dapat diterapkan secara *fast-forward*, yaitu riwayat lokal merupakan kelanjutan langsung dari riwayat remote. Jika remote memiliki commit yang belum ada di lokal, *push* ditolak (*non-fast-forward*) dan pengguna perlu mengambil perubahan terlebih dahulu. *Pull* mengambil commit baru dari remote dan menggabungkannya ke *branch* yang sedang aktif, sehingga secara konsep *pull* merupakan kombinasi *fetch* dan *merge* [1].

## E. Istilah Penting

Istilah yang digunakan dalam artikel ini dirangkum pada Tabel 1.

**Tabel 1.** Istilah penting dalam Git dan GitHub

| Istilah | Penjelasan |
|---|---|
| Git | Aplikasi kontrol versi yang berjalan di komputer lokal untuk mencatat riwayat perubahan berkas. |
| GitHub | Layanan hosting repositori Git di internet untuk menyimpan, berbagi, dan berkolaborasi. |
| Repository | Folder proyek beserta seluruh riwayat perubahannya. |
| Local repository | Repositori yang berada di komputer pengguna. |
| Remote | Repositori yang berada di server, misalnya GitHub; nama standarnya origin. |
| Commit | Snapshot kondisi proyek pada satu titik waktu, disertai pesan penjelasan. |
| Branch | Cabang pengembangan; cabang utama umumnya bernama main. |
| Push | Mengirim commit dari repositori lokal ke GitHub. |

| Istilah | Penjelasan |
|---|---|
| Pull | Mengambil commit terbaru dari GitHub dan menggabungkannya ke repositori |
| lokal. Clone | Menyalin seluruh repositori dari GitHub ke komputer untuk pertama kali. |
| Staging area | Area transit tempat berkas ditandai dengan git addsebelum di-commit. |

## III. METODE

### A. Pendekatan

Penelitian ini menggunakan studi pustaka terhadap dokumentasi resmi Git dan GitHub [8], [4] serta literatur terkait [1], [2], [5]–[7]. Hasil studi disusun menjadi prosedur operasional yang berurutan, kemudian diilustrasikan pada sebuah studi kasus pembuatan dan publikasi website portfolio.

### B. Alat dan Bahan

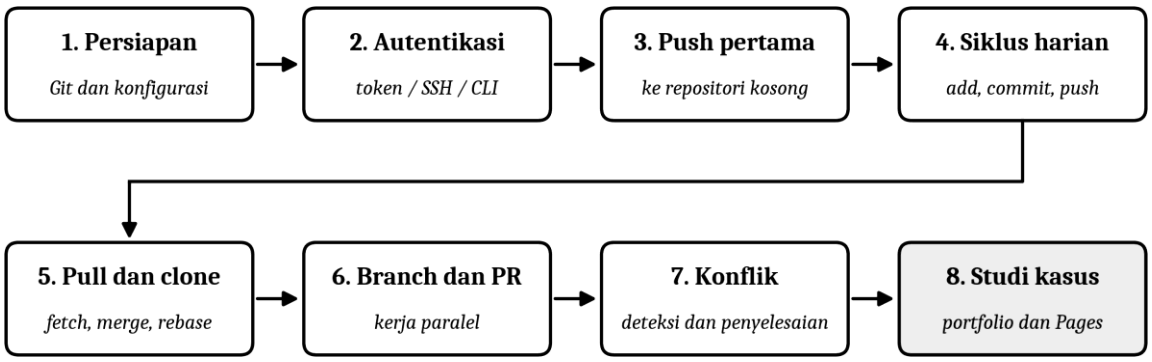
Komponen yang diperlukan tercantum pada Tabel 2.

**Tabel 2.** Alat dan bahan

| Komponen | Spesifikasi | Keterangan |
|---|---|---|
| Git | Versi 2.28 atau lebih baru | Opsi init.defaultBranchtersedia sejak versi ini. |
| Terminal | Git Bash, Terminal, atau Command Prompt | Tempat menjalankan perintah. |
| Editor teks | Visual Studio Code (disarankan) | Menulis dan mengedit berkas proyek. |
| Peramban web | Peramban modern | Menguji tampilan dan mengakses GitHub. |
| Akun GitHub | Akun gratis | Menyimpan repositori jarak jauh. |
| Repositori | faridSrydi/github-pcd (publik, kosong) | Tujuan *push* pertama. |
| Python 3 | Opsional | Menjalankan server lokal dengan python -m http.server. |

### C. Tahapan Penelitian

Prosedur disusun dalam delapan tahap yang saling berurutan seperti pada Gambar 2. Tahap 1 sampai 7 membahas operasi Git dan GitHub secara umum, sedangkan tahap 8 menerapkannya pada studi kasus.




**Gambar 2.** Tahapan penyusunan dan penerapan prosedur.

### D. Catatan Penyajian

Keluaran terminal pada artikel ini bersifat ilustratif: nilai *hash* commit, jumlah objek, versi Git, dan jalur direktori akan berbeda pada setiap komputer. Blok perintah ditampilkan dalam bentuk gambar terminal, sedangkan nama perintah yang muncul di dalam kalimat dan tabel ditulis dengan huruf monospasi.

## IV. HASIL DAN PEMBAHASAN

### A. Persiapan Lingkungan Kerja

Git dipasang sesuai sistem operasi. Pada Windows, pemasangan dilakukan melalui *installer* dari situs resmi Git yang sekaligus menyertakan Git Bash dan Git Credential Manager. Pada macOS digunakan `brew install git`, dan pada Linux berbasis Debian digunakan `sudo apt install git` [8]. Versi diperiksa dengan `git --version`, lalu identitas pengguna diatur satu kali menggunakan `git config --global` seperti pada Gambar 3.




**Gambar 3.** Pemeriksaan versi dan konfigurasi identitas Git.

Pengaturan `user.name` dan `user.email` dicatat pada setiap commit sebagai identitas penulis. Alamat email sebaiknya sama dengan email akun GitHub agar commit terhubung ke profil. Pengaturan `init.defaultBranch main` membuat repositori baru memakai *branch* `main`, dan opsi `--global` memberlakukan pengaturan pada semua repositori di komputer tersebut.

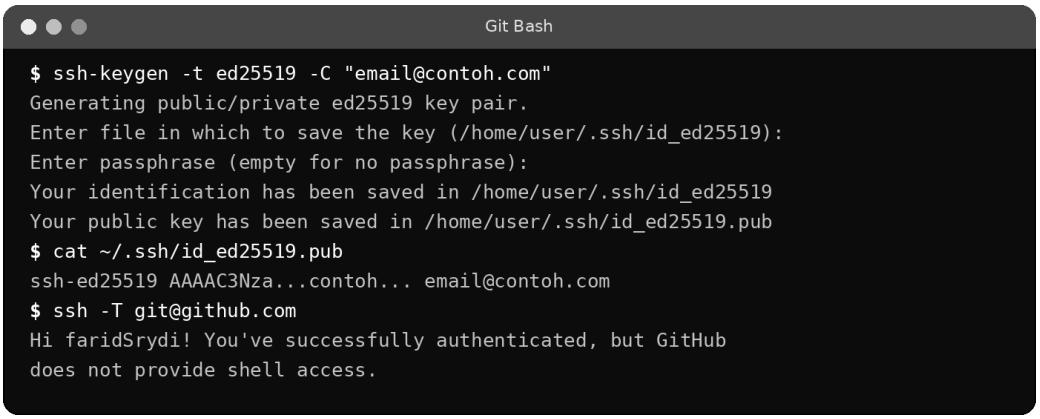
### B. Autentikasi ke GitHub

Sejak tahun 2021, GitHub tidak lagi menerima kata sandi akun untuk operasi Git melalui HTTPS [4]. Pengguna dapat memilih salah satu dari tiga metode pada Tabel 3.

**Tabel 3.** Perbandingan metode autentikasi

| Metode | Cara kerja | Kelebihan | Kekurangan |
|---|---|---|---|
| *Personal access token* (HTTPS) | Token dibuat di *Settings* > *Developer settings*, lalu dipakai sebagai pengganti kata sandi. | Mudah disiapkan; tidak bergantung pada port SSH. | Token harus dijaga kerahasiaannya dan dapat kedaluwarsa. |
| Kunci SSH | Pasangan kunci dibuat dengan ssh-keygen; kunci publik didaftarkan di GitHub; URL remote berbentuk `git@github.com:...`. | Tidak perlu memasukkan kredensial berulang kali. | Persiapan awal lebih panjang. |
| GitHub CLI | Perintah `gh auth login` menjalankan proses login melalui peramban. | Paling ringkas bagi pemula. | Memerlukan pemasangan GitHub CLI. |

Pada metode token, pengguna membuat token dengan izin baca dan tulis pada isi repositori (*Contents: Read and write* untuk *fine-grained token*, atau cakupan *repo* untuk *classic token*), menyalinnya, lalu menempelkannya pada kolom kata sandi ketika Git meminta kredensial. Token hanya ditampilkan satu kali sehingga harus disimpan di tempat yang aman. Pada metode SSH, pembuatan kunci dan pengujian koneksi ditunjukkan pada Gambar 4. Apa pun metodenya, token dan kunci privat tidak boleh dimasukkan ke dalam repositori.




**Gambar 4.** Pembuatan kunci SSH dan pengujian koneksi ke GitHub.

## C. Push Pertama ke Repositori Kosong

Repositori github-pcd yang masih kosong dapat langsung diisi dari proyek lokal. Urutan langkah beserta fungsinya dirangkum pada Tabel 4.

**Tabel 4.** Langkah *push* pertama ke repositori kosong

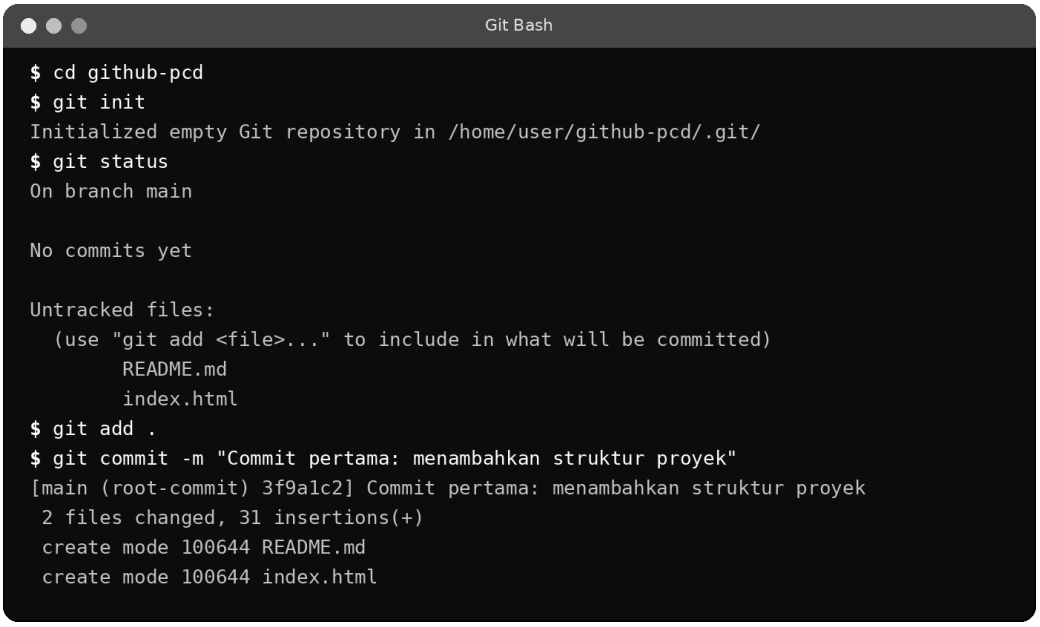
| No. | Perintah | Fungsi |
|---|---|---|
| 1 | cd github-pcd | Masuk ke folder proyek. |
| 2 | git init | Membuat folder tersembunyi .gityang menyimpan riwayat dan konfigurasi; dilakukan sekali per proyek. |
| 3 | (membuat .gitignore) | Mendaftar berkas atau folder yang tidak boleh dilacak, misalnya .envdan node_modules/. |
| 4 | git status | Menampilkan berkas yang belum dilacak, berubah, atau sudah di-*staging*. |
| 5 | git add . | Memasukkan semua perubahan ke *staging area*. |
| 6 | git commit -m "pesan" | Menyimpan *snapshot* dari *staging* ke riwayat lokal beserta pesannya. |
| 7 | git branch -M main | Memastikan nama *branch* adalah main. |
| 8 | git remote add origin <url> | Mendaftarkan alamat repositori GitHub dengan nama origin. |
| 9 | git push -u origin main | Mengirim commit ke GitHub dan mengaitkan mainlokal dengan origin/main(*upstream*). |

Berkas .gitignore(Gambar 5) penting agar berkas sampah, dependensi berukuran besar, dan data rahasia seperti kunci API tidak ikut terunggah. Isi berkas ini ditulis sebelum perintah git add
.dijalankan.

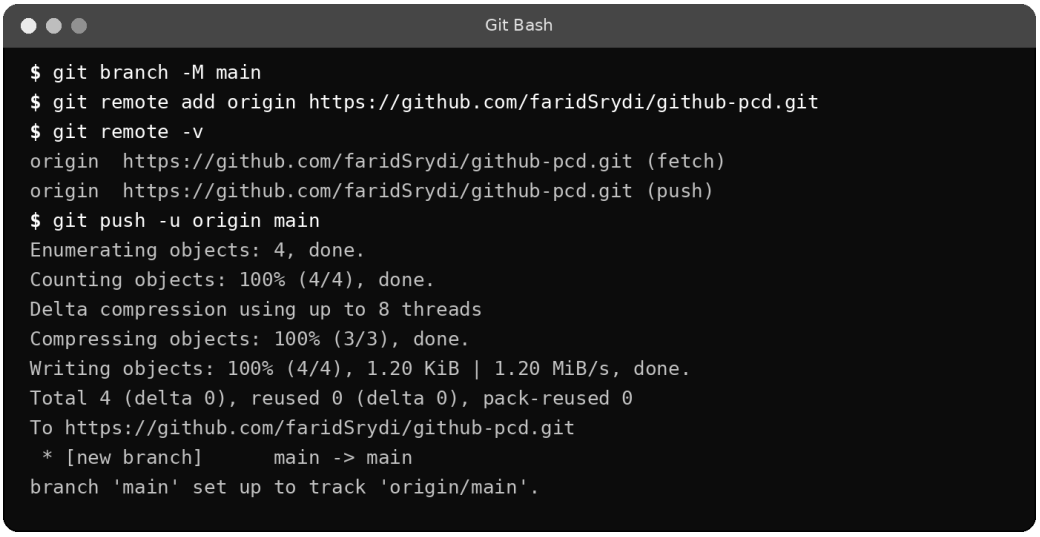



**Gambar 5.** Contoh isi berkas *.gitignore* untuk proyek Python dan Node.

Gambar 6 dan Gambar 7 memperlihatkan contoh keluaran terminal untuk langkah 1 sampai 9.




**Gambar 6.** Inisialisasi repositori, pemeriksaan status, *staging*, dan commit pertama.

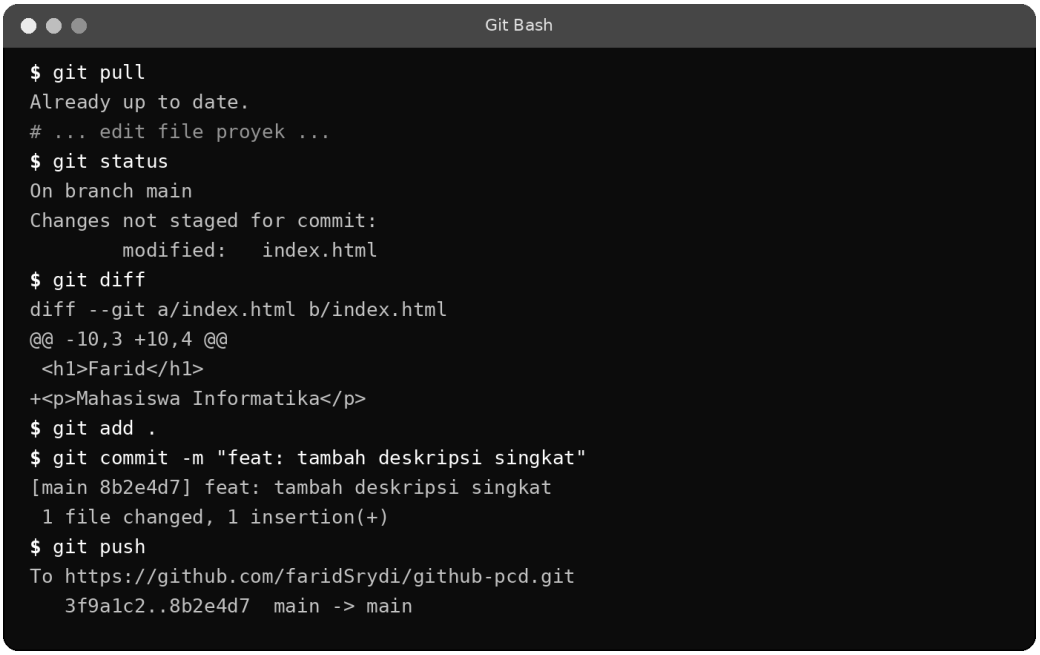



**Gambar 7.** Menghubungkan remote dan mengirim commit pertama ke GitHub.

Opsi -u (*set upstream*) pada langkah terakhir menyimpan hubungan antara *branch* lokal dan *branch* remote, sehingga perintah berikutnya cukup git push dan git pull tanpa argumen tambahan. Setelah *push* berhasil, berkas proyek tampil pada halaman repositori di GitHub.

### D. Siklus Kerja Harian

Setelah *push* pertama, pekerjaan sehari-hari mengikuti siklus yang sama (Gambar 8): mengambil perubahan terbaru dengan git pull, mengedit berkas, memeriksa perubahan dengan git status dan git diff, memasukkannya ke *staging* dengan git add, menyimpannya dengan git commit, lalu mengirimnya dengan git push. Memulai dengan *pull* mengurangi kemungkinan *push* ditolak karena remote sudah lebih maju.




**Gambar 8.** Siklus kerja harian: *pull*, ubah, *commit*, dan *push*.

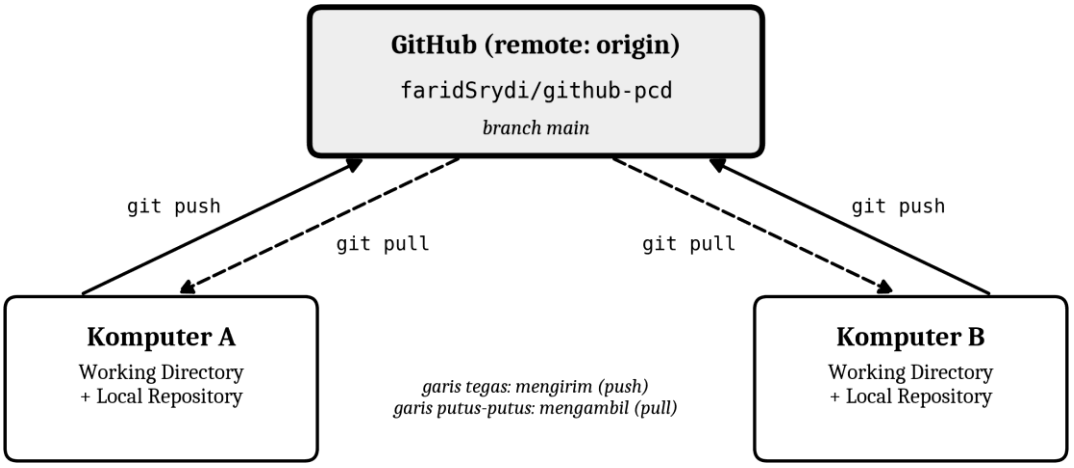
Commit sebaiknya kecil dan berfokus pada satu tujuan agar riwayat mudah dibaca dan dibatalkan. Pesan commit dapat mengikuti konvensi sederhana pada Tabel 5 yang sejalan dengan gagasan *Conventional Commits* [9].

**Tabel 5.** Jenis pesan commit yang umum digunakan

| Jenis | Kegunaan | Contoh |
|---|---|---|
| feat | Fitur baru | feat: tambah halaman registrasi |
| fix | Perbaikan *bug* | fix: perbaiki validasi email |
| docs | Dokumentasi | docs: perbarui README |
| style | Perubahan tampilan atau format tanpa mengubah logika | style: tambah mode gelap |
| refactor | Merapikan kode tanpa mengubah fungsi | refactor: pisahkan fungsi koneksi chore |
| | Tugas kecil atau konfigurasi | chore: tambah .gitignore |

## E. Pull, Fetch, dan Clone

Perintah git clone mengunduh seluruh repositori beserta riwayatnya ke komputer baru dan otomatis menetapkan remote origin. Perintah git pull pada dasarnya merupakan gabungan git fetch, yang mengunduh commit baru dari remote, dan git merge, yang menggabungkannya ke *branch* aktif [1]. Alur kerja dua komputer dengan satu remote ditunjukkan pada Gambar 9, sedangkan perbedaan fetch dan pull diringkas pada Tabel 6.




**Gambar 9.** Dua komputer yang berbagi satu repositori di GitHub melalui *push* dan *pull*.

**Tabel 6.** Perbandingan git fetch dan git pull

| **Aspek** | git fetch | git pull |
|---|---|---|
| Mengunduh perubahan dari remote | Ya | Ya |
| Menggabungkan ke *branch* lokal | Tidak | Ya |
| Risiko mengubah berkas lokal | Tidak ada | Ada (dapat menimbulkan konflik) |
| Kapan digunakan | Ingin meninjau perubahan remote terlebih dahulu | Ingin langsung menyinkronkan |

Gambar 10 memperlihatkan perintah untuk *clone*, peninjauan dengan fetch, penggabungan, dan pull --rebase. Opsi --rebase meletakkan ulang commit lokal di atas commit terbaru dari remote sehingga riwayat berbentuk garis lurus tanpa commit *merge* tambahan. Rebase sebaiknya hanya diterapkan pada commit lokal yang belum dibagikan kepada orang lain [1].

```
Git Bash
$ git clone https://github.com/faridSrydi/github-pcd.git
Cloning into 'github-pcd'...
remote: Enumerating objects: 7, done.
Receiving objects: 100% (7/7), done.
$ cd github-pcd

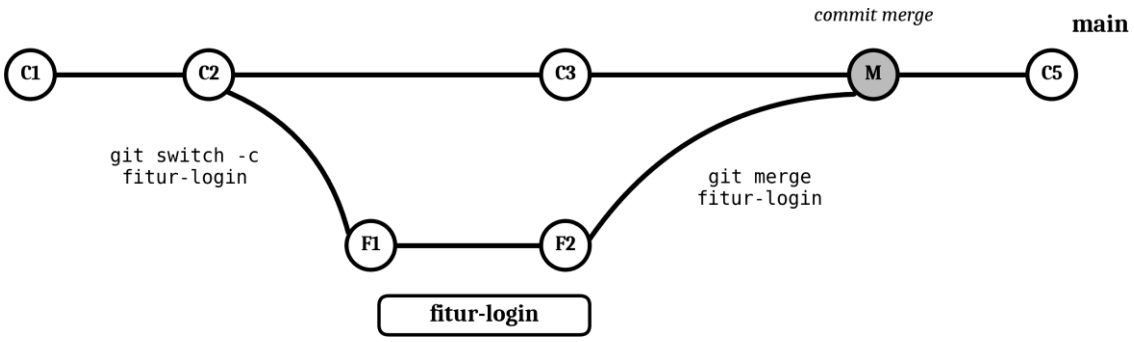
# Meninjau dulu sebelum menggabungkan
$ git fetch origin
$ git log HEAD..origin/main --oneline
8b2e4d7 feat: tambah deskripsi singkat
$ git merge origin/main
Updating 3f9a1c2..8b2e4d7
Fast-forward
 index.html | 1 +
 1 file changed, 1 insertion(+)

# Atau langsung sinkron dengan riwayat yang lurus
$ git pull --rebase
```

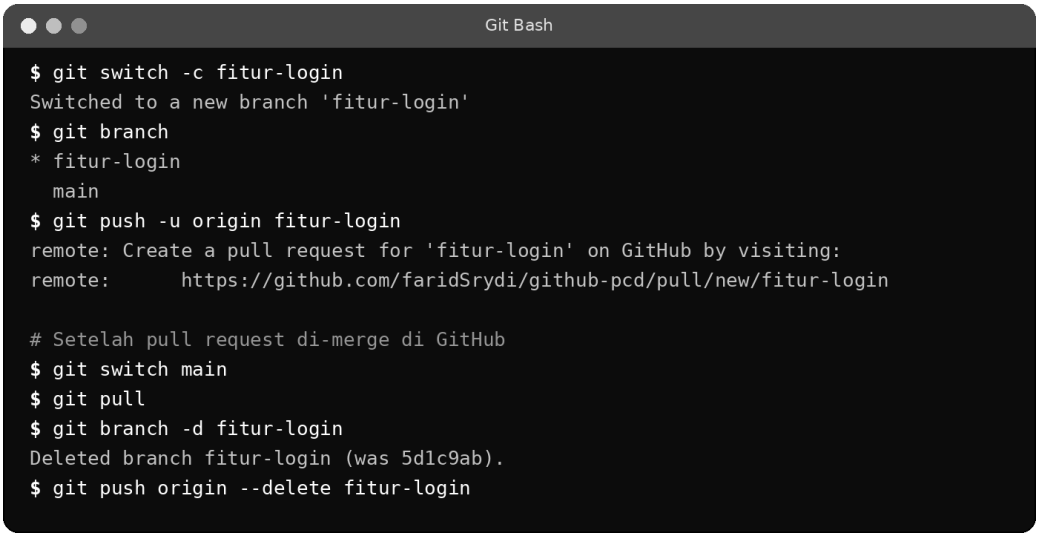
**Gambar 10.** *Clone*, *fetch*, *merge*, dan *pull --rebase*.

## F. Branch dan Pull Request

*Branch* memungkinkan pengerjaan fitur tanpa mengganggu *branch* utama. Gambar 11 mengilustrasikan cabang fitur-login yang dibuat dari commit C2, dikembangkan melalui commit F1 dan F2, lalu digabungkan kembali menjadi commit *merge* M. Perintah yang digunakan ditunjukkan pada Gambar 12.




**Gambar 11.** Pembuatan cabang *fitur-login* dan penggabungannya ke *main*.




**Gambar 12.** Perintah untuk membuat, mengirim, dan menghapus *branch*.

Perintah `git branch -d` hanya menghapus *branch* yang sudah digabungkan sehingga lebih aman daripada `-D`. *Pull request* (PR) adalah mekanisme GitHub untuk mengusulkan penggabungan sebuah *branch* ke `main` melalui peninjauan [4]. Alurnya adalah sebagai berikut.

1. Mengirim *branch* fitur dengan `git push -u origin fitur-login`.
2. Membuka repositori di GitHub dan menekan tombol *Compare & pull request*.
3. Mengisi judul dan deskripsi perubahan, lalu membuat *pull request*.
4. Setelah ditinjau dan disetujui, menekan *Merge pull request*.
5. Menyinkronkan kembali repositori lokal dengan `git switch main` dan `git pull`.

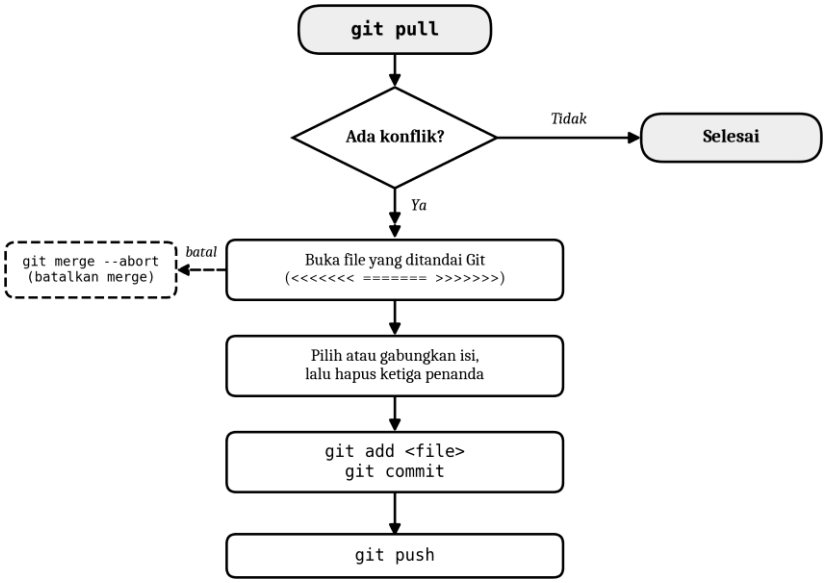
## G. Penanganan Konflik

Konflik terjadi ketika dua perubahan memodifikasi baris yang sama pada berkas yang sama sehingga Git tidak dapat menggabungkannya secara otomatis. Git menandai bagian bermasalah di dalam berkas dengan tiga penanda (Gambar 13): bagian di atas `=======` adalah versi lokal (`HEAD`), dan bagian di bawahnya adalah versi dari remote.




**Gambar 13.** Penanda konflik yang ditambahkan Git pada berkas.

Langkah penyelesaian ditunjukkan pada diagram alir (Gambar 14) dan perintahnya pada Gambar 15. Pengguna memilih salah satu versi atau menggabungkan keduanya, menghapus ketiga penanda, menandai berkas sebagai selesai dengan `git add`, lalu membuat commit. Apabila ingin membatalkan seluruh proses penggabungan, `git merge --abort` mengembalikan kondisi ke sebelum `git pull`.




**Gambar 14.** Diagram alir penyelesaian konflik penggabungan.

Git Bash

```
$ git pull
Auto-merging index.html
CONFLICT (content): Merge conflict in index.html
Automatic merge failed; fix conflicts and then commit the result.

# Setelah konflik diedit dan penanda dihapus
$ git add index.html
$ git commit -m "fix: selesaikan konflik pada index.html"
$ git push

# Untuk membatalkan proses merge
$ git merge --abort
```

**Gambar 15.** Pesan konflik saat *pull* dan perintah penyelesaiannya.

## H. Perintah Pendukung

Selain perintah utama, beberapa perintah berikut membantu menelusuri dan memperbaiki riwayat (Tabel 7).

**Tabel 7.** Perintah pendukung dalam penggunaan Git

| Perintah | Fungsi |
|---|---|
| git log --oneline --graph | Melihat riwayat commit secara ringkas dan bergrafik. |
| git show <id-commit> | Melihat detail satu commit. |
| git restore <file> | Membatalkan perubahan berkas yang belum di-*staging*. |
| git restore --staged <file> | Mengeluarkan berkas dari *staging* tanpa menghapus perubahan. |
| git commit --amend | Mengubah commit terakhir; hanya untuk commit yang belum di-*push*. |
| git revert <id-commit> | Membatalkan commit dengan membuat commit baru; aman untuk riwayat publik. |
| git stash/ git stash pop | Menyimpan sementara perubahan yang belum di-commit dan mengembalikannya. |

| Perintah | Fungsi |
|---|---|
| git remote -v | Melihat alamat remote. |
| git remote set-url origin <url> | Mengganti alamat remote. |
| git rm --cached <file> | Berhenti melacak berkas tanpa menghapusnya dari komputer. |

## I. Studi Kasus: Website Portfolio

Studi kasus ini menerapkan prosedur sebelumnya pada pembuatan website portfolio statis yang disimpan di repositori github-pcd dan dipublikasikan melalui GitHub Pages [10]. Seluruh kode sumber disertakan pada Lampiran B.

### *1) Struktur proyek*

Proyek terdiri atas empat berkas pada direktori utama (Gambar 16). Berkas halaman utama harus bernama index.html karena server web, termasuk GitHub Pages, memuatnya secara otomatis sebagai halaman depan. Peran masing-masing berkas dijelaskan pada Tabel 8.



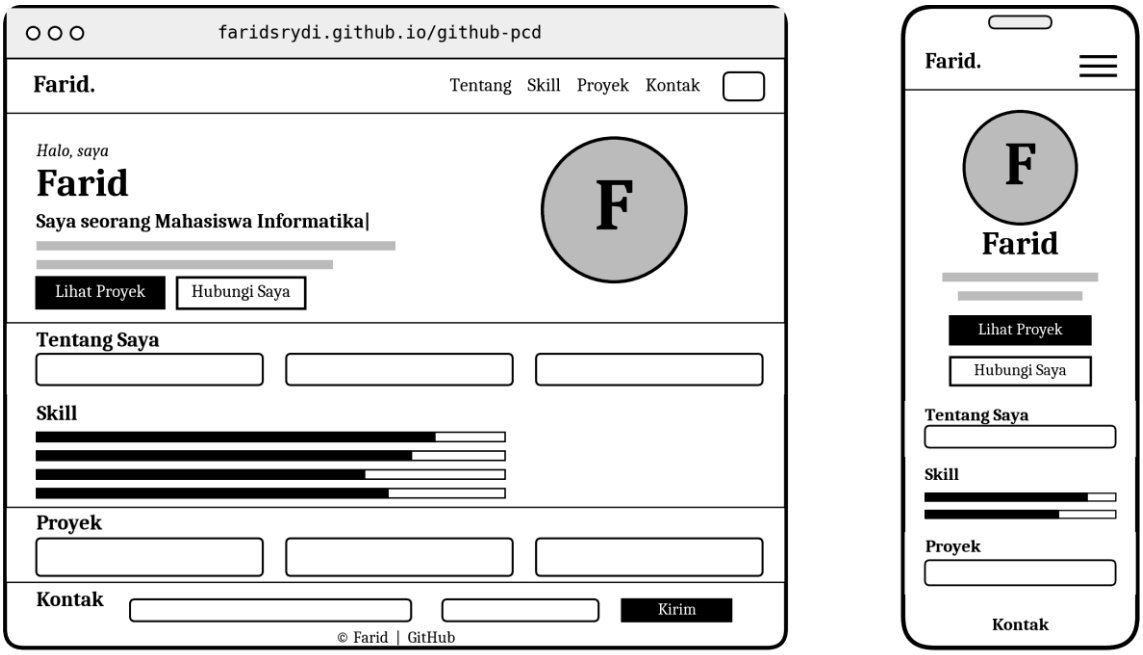

**Gambar 16.** Struktur direktori proyek portfolio.

**Tabel 8.** Berkas pada proyek portfolio

| Berkas | Peran |
|---|---|
| index.html | Kerangka dan isi halaman: navigasi, *hero*, tentang, skill, proyek, dan kontak. style.css |
| | Tampilan: variabel warna, tema gelap, tata letak *grid* dan *flexbox*, serta aturan |
| responsif. script.js | Interaksi: pengganti tema, menu HP, efek ketik, animasi *scroll*, dan form kontak. |
| README.md | Dokumentasi proyek, termasuk laporan *push* dan *pull* ini. |

### *2) Fitur dan cara kerja*

Halaman dirancang responsif sehingga menyesuaikan lebar layar (Gambar 17). Pada layar selebar 800 piksel atau kurang, menu berubah menjadi tombol *hamburger* dan bagian *hero* menjadi satu kolom. Fitur interaktif yang diimplementasikan pada script.js dirangkum pada Tabel 9; deteksi elemen yang masuk layar memanfaatkan antarmuka *IntersectionObserver* bawaan peramban [11].




*(a) Tampilan desktop* *(b) Tampilan HP*

**Gambar 17.** Rancangan tata letak (*wireframe*) halaman portfolio pada tampilan desktop dan HP.

**Tabel 9.** Fitur interaktif pada script.js

| No. | Fitur | Cara kerja |
|---|---|---|
| 1 | Tahun footer | new Date().getFullYear()mengisi tahun secara otomatis. |
| 2 | Mode gelap/terang | Mengubah atribut data-themepada elemen html, lalu menyimpan pilihan ke localStorageagar diingat saat halaman dibuka kembali. |
| 3 | Menu HP | Menambah atau menghapus kelas openpada menu setiap tombol diklik. |
| 4 | Efek ketik | Fungsi type()menambah atau menghapus satu huruf dengan setTimeout, bergantian di antara isi larik roles. |
| 5 | Animasi *scroll* | IntersectionObservermendeteksi elemen yang masuk layar, menambahkan kelas visible, dan mengisi lebar bar skill dari atribut data-level. |
| 6 | Menu aktif | Observer kedua menandai tautan menu sesuai bagian yang sedang dilihat. |
| 7 | Form kontak | Menyusun tautan mailto:dari isi form sehingga tidak memerlukan server. |

Sebelum dipublikasikan, konten contoh perlu diganti: data diri, tingkat skill (dua tempat per skill, yaitu angka data-level dan teks persentase), kartu proyek, isi larik roles, alamat email tujuan EMAIL_TUJUANpada form kontak, dan warna utama pada variabel CSS --primary.

### *3) Pengujian lokal*

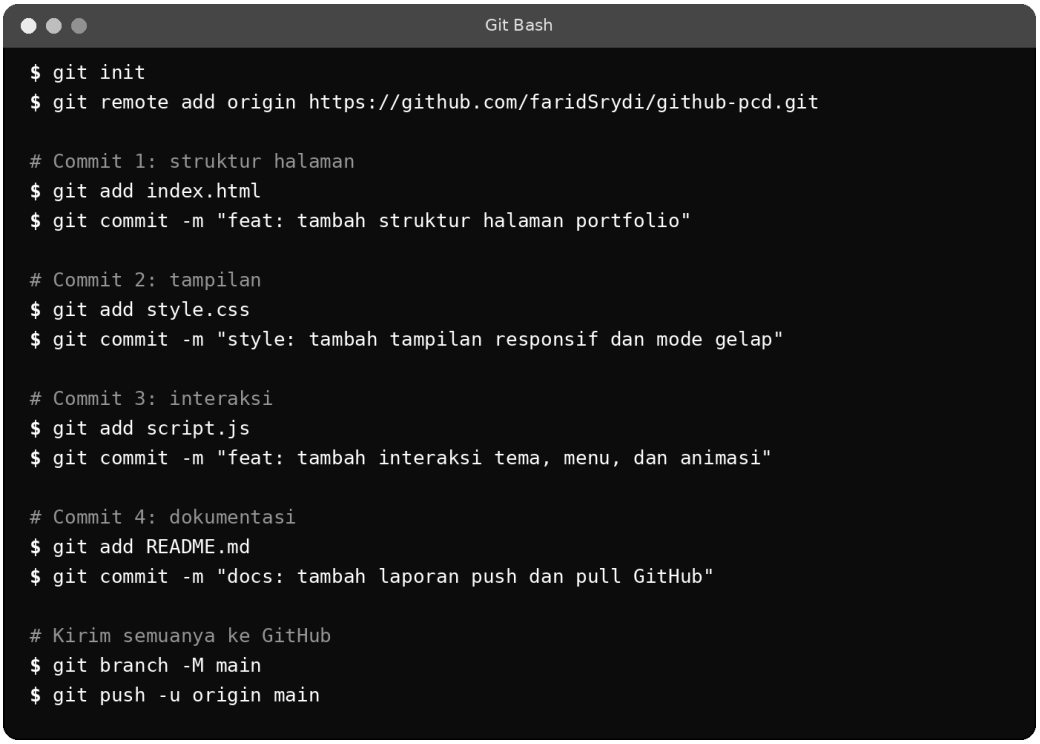
Sebelum di-commit, halaman diuji dengan membuka index.html di peramban atau menjalankan server lokal (Gambar 18) lalu membuka alamat http://localhost:8000. Hal yang diperiksa meliputi kebenaran tautan menu, fungsi tombol tema, kerapian tampilan pada layar kecil (melalui *device toolbar* peramban dengan tombol F12), dan tidak adanya pesan galat pada konsol.



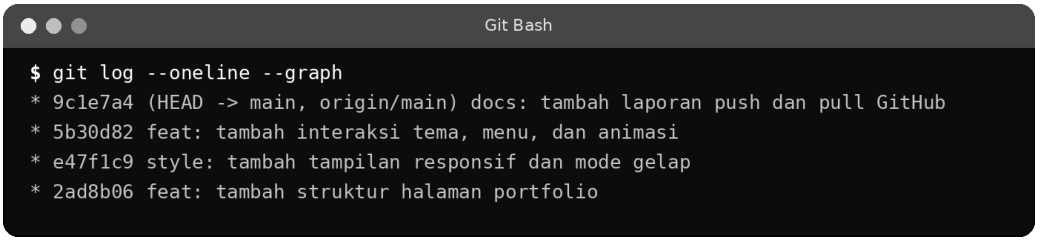

**Gambar 18.** Menjalankan server lokal untuk menguji halaman.

### *4) Commit bertahap dan push*

Pekerjaan dipecah menjadi empat commit kecil, masing-masing berisi satu tujuan, dengan prefiks pesan sesuai Tabel 5. Berkas ditambahkan satu per satu dengan `git add <nama-berkas>` agar setiap commit hanya memuat berkas yang relevan (Gambar 19). Perintah `git branch -M main` dijalankan setelah ada commit agar nama *branch* pasti `main`, lalu `git push -u origin main` mengirim keempat commit sekaligus. Riwayat hasilnya dapat diperiksa dengan `git log --oneline --graph` (Gambar 20).




**Gambar 19.** Empat commit bertahap dan *push* ke repositori github-pcd.



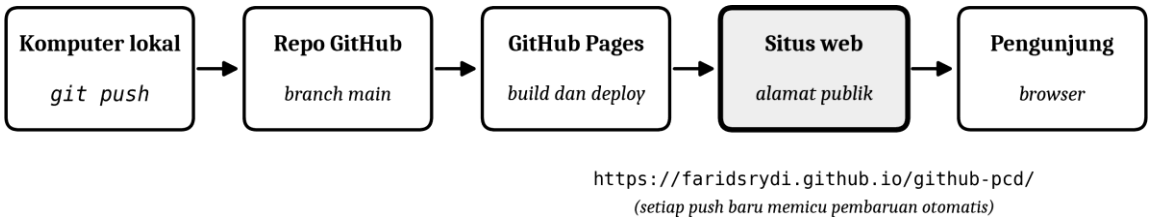

**Gambar 20.** Riwayat commit hasil studi kasus (contoh keluaran).

### *5) Publikasi dengan GitHub Pages*

GitHub Pages menyajikan berkas statis langsung dari repositori dan tersedia gratis untuk repositori publik [10]. Langkah pengaktifannya adalah sebagai berikut.

1. Membuka repositori di GitHub, lalu memilih *Settings*.
2. Memilih menu *Pages* pada panel kiri.
3. Pada *Build and deployment*, memilih *Source* berupa *Deploy from a branch*.
4. Memilih *branch* maindan folder */ (root)*, lalu menekan *Save*.
5. Menunggu sekitar satu sampai tiga menit hingga alamat situs tampil pada halaman Pages.

Alamat situs berpola https://<username>.github.io/<nama-repo>/, sehingga untuk repositori ini menjadi https://faridsrydi.github.io/github-pcd/ (Gambar 21). Setiap *push* baru ke main memicu pembaruan situs secara otomatis, dan prosesnya dapat dipantau pada tab *Actions*. Pembaruan portfolio berikutnya cukup mengikuti siklus harian pada Gambar 8.




**Gambar 21.** Alur dari *push* hingga situs dapat diakses pengunjung.

## J. Pemecahan Masalah

Kendala yang paling sering dijumpai beserta penyebab dan solusinya dirangkum pada Tabel 10.

**Tabel 10.** Masalah umum dan solusinya

| Pesan galat / masalah | Penyebab | Solusi |
|---|---|---|
| fatal: not a git repository | Folder belum diinisialisasi. | Jalankan git initatau pindah ke folder repositori yang benar. |
| remote origin already exists | Remote sudah pernah ditambahkan. | git remote set-url origin <url> |
| Authentication failed | Memakai kata sandi akun. | Gunakan token, kunci SSH, atau gh auth login. |
| src refspec main does not match any | Belum ada commit, atau nama *branch* bukan main. | Lakukan git commit, lalu git branch -M main. |
| rejected (non-fast-forward) | Remote memiliki commit yang belum ada di lokal. | git pull --rebase, lalu git push. |
| refusing to merge unrelated histories | Riwayat lokal dan remote berasal dari awal yang berbeda. | git pull origin main --allow-unrelated-histories |
| Permission denied (publickey) | Kunci SSH belum terdaftar di GitHub. | Tambahkan kunci publik pada *Settings* > *SSH and GPG keys*. |
| Berkas lebih dari 100 MB ditolak | Batas ukuran berkas GitHub. | Gunakan Git LFS atau jangan meng-commit berkas tersebut. |
| .envatau token terlanjur ter-*push* | Berkas rahasia ikut di-commit. | Segera cabut dan ganti kredensial, lalu hapus dari riwayat; menghapus pada commit baru saja tidak cukup. |
| CSS tidak termuat di GitHub Pages | Nama berkas berbeda huruf besar/kecil; server membedakannya. | Samakan penulisan nama berkas; gunakan git mv Style.css style.css. |

| Pesan galat / masalah | Penyebab | Solusi |
|---|---|---|
| Halaman 404 pada alamat Pages | Berkas utama bukan index.html, atau proses *build* belum selesai. | Pastikan index.html ada di direktori utama, tunggu beberapa menit, dan cek tab *Actions*. |
| Perubahan tidak tampil di situs | *Cache* peramban, atau lupa melakukan *push*. | Tekan Ctrl+F5; periksa git statusdan git log. |

### K. Praktik Terbaik

1. *Pull* sebelum mulai bekerja dan sebelum *push* untuk meminimalkan konflik.
2. Commit sering dengan ukuran kecil dan pesan yang jelas.
3. Jangan pernah meng-commit data rahasia seperti kata sandi, token, kunci API, atau berkas
   .env; daftarkan pada .gitignore.
4. Gunakan *branch* untuk fitur atau perbaikan agar maintetap stabil.
5. Hindari git push --force pada *branch* bersama karena menimpa riwayat di remote; jika terpaksa, gunakan --force-with-lease.
6. Tulis README.mdyang menjelaskan tujuan proyek dan cara menjalankannya.
7. Periksa git statussebelum setiap commitdan push.

## V. KESIMPULAN

Artikel ini menyajikan prosedur lengkap *push* dan *pull* pada GitHub, mulai dari persiapan lingkungan hingga penanganan konflik dan publikasi situs. Pembahasan menunjukkan bahwa operasi tersebut dapat dikuasai dengan memahami tiga hal: alur perubahan dari *working directory* ke *staging area*, repositori lokal, dan remote; hubungan antara *branch* lokal dan *branch* remote; serta cara menangani penolakan *push* dan konflik penggabungan. Studi kasus website portfolio memperlihatkan bahwa prosedur yang sama berlaku dari *push* pertama ke repositori kosong hingga publikasi melalui GitHub Pages.

Pengembangan selanjutnya dapat mencakup penggunaan GitHub Actions untuk otomatisasi pengujian dan *deployment*, aturan proteksi *branch*, serta alur kolaborasi untuk tim yang lebih besar. Pembaca juga disarankan merujuk dokumentasi resmi Git dan GitHub untuk fitur terbaru, karena antarmuka dan kebijakan layanan dapat berubah.

## DAFTAR PUSTAKA


[1] S. Chacon and B. Straub, *Pro Git*, 2nd ed. New York, NY, USA: Apress, 2014. [Online]. Available: https://git-scm.com/book

[2] J. Loeliger and M. McCullough, *Version Control with Git*, 2nd ed. Sebastopol, CA, USA: O'Reilly Media, 2012.

[3] D. Spinellis, "Git," *IEEE Softw.*, vol. 29, no. 3, pp. 100–101, 2012.

[4] GitHub, "GitHub Docs," GitHub, Inc. Accessed: Nov. 3, 2026. [Online]. Available: https://docs.github.com

[5] J. D. Blischak, E. R. Davenport, and G. Wilson, "A quick introduction to version control with Git and GitHub," *PLoS Comput. Biol.*, vol. 12, no. 1, p. e1004668, 2016.

[6] Y. Perez-Riverol *et al.*, "Ten simple rules for taking advantage of Git and GitHub," *PLoS Comput. Biol.*, vol. 12, no. 7, p. e1004947, 2016.

[7] K. Ram, "Git can facilitate greater reproducibility and increased transparency in science," *Source Code Biol. Med.*, vol. 8, no. 1, p. 7, 2013.

[8] Git, “Git documentation.” Accessed: Nov. 3, 2026. [Online]. Available: https://git-scm.com/doc

[9] “Conventional Commits.” Accessed: Nov. 3, 2026. [Online]. Available: https://www.conventionalcommits.org

[10] GitHub, “GitHub Pages,” GitHub, Inc. Accessed: Nov. 3, 2026. [Online]. Available: https://pages.github.com

[11] Mozilla, “MDN Web Docs.” Accessed: Nov. 3, 2026. [Online]. Available: https://developer.mozilla.org